Cafeteria Order and Billing System

Phase 3 — Database and DAO Layer

Object Oriented Programming (CSEG1044)

1. Introduction

Phase 3 extends the Cafeteria Order and Billing System by implementing the full database and DAO layer. Building on the package design from Phase 2, this phase adds the four new entity models (Customer, Order, OrderItem, Bill), four new DAO classes, and two new service classes (OrderService and BillService) to complete all core data workflows.

After Phase 3, the application can: authenticate users, manage the menu, place orders for customers, and generate itemised bills with tax — all backed by a live MySQL database.

1.1 Project Phase Progress

Phase	Title	Status
0	Proposal and Backlog	✅Complete
1	Requirements and Use Cases	✅Complete
2	UML and Package Design	✅Complete
3	Database and DAO Layer	✅Complete
4	UI and Integration	⬜ Pending
5	Testing, Build, and Demo	⬜ Pending

2. What's New in Phase 3

The table below lists every file in the project. Files marked ★ were added in this phase.

File	Layer	Phase Added
`Main.java`	Entry Point	Phase 2
`User.java`	Model	Phase 2
`MenuItem.java`	Model	Phase 2
`Customer.java`	Model	Phase 3 ★
`Order.java`	Model	Phase 3 ★
`OrderItem.java`	Model	Phase 3 ★
`Bill.java`	Model	Phase 3 ★
`DBConnection.java`	Util	Phase 2
`UserDao.java`	DAO	Phase 2
`MenuDao.java`	DAO	Phase 2
`CustomerDao.java`	DAO	Phase 3 ★
`OrderDao.java`	DAO	Phase 3 ★
`OrderItemDao.java`	DAO	Phase 3 ★
`BillDao.java`	DAO	Phase 3 ★
`UserService.java`	Service	Phase 2
`MenuService.java`	Service	Phase 2
`OrderService.java`	Service	Phase 3 ★
`BillService.java`	Service	Phase 3 ★
`LoginFrame.java`	UI	Phase 2
`AdminDashboard.java`	UI	Phase 2
`OperatorDashboard.java`	UI	Phase 2
`MenuManagementPanel.java`	UI	Phase 2
`OrderHistoryPanel.java`	UI — stub	Phase 4
`SalesSummaryPanel.java`	UI — stub	Phase 4
`PlaceOrderPanel.java`	UI — stub	Phase 4

3. Architecture

3.1 3-Layer Call Flow

The call flow is unchanged from Phase 2. No UI class ever calls a DAO directly.

```
PlaceOrderPanel → OrderService.placeOrder() → CustomerDao / OrderDao / OrderItemDao
PlaceOrderPanel → BillService.generateBill() → BillDao + OrderService.markAsBilled()
OrderHistoryPanel → OrderService.getAllOrders() → OrderDao + OrderItemDao
```

3.2 Order Placement Sequence

When an operator places an order, OrderService coordinates three DAOs in a fixed sequence:

- Step 1 — CustomerDao.findOrCreate(): look up the customer by name, or insert a new record.
- Step 2 — OrderDao.insert(): insert the order header and obtain the auto-generated order_id.
- Step 3 — OrderItemDao.insert() (loop): insert each line item, setting the real order_id on each.

3.3 Bill Generation Sequence

BillService.generateBill() receives an Order object that already has its items loaded in memory:

- Calculate subtotal by summing quantity × unitPrice across all items.
- Apply 5% tax rate (TAX_RATE constant in BillService) to get taxAmount.
- Persist the Bill via BillDao.insert() and capture the generated bill_id.
- Call OrderService.markAsBilled() to update the order status to BILLED.

3.4 Snapshot Pricing

Each OrderItem stores its own unit_price at the time of ordering. This means that if an admin later changes a menu item's price, all historical orders and bills are unaffected. The unit_price in order_items is a snapshot, not a live foreign-key reference to the current price.

3.5 DBConnection — Singleton

All DAOs share a single JDBC connection through DBConnection.getInstance(). The connection is lazily created on first access and automatically re-established if it is found to be closed. The credentials are configured as constants in DBConnection.java:

```
URL:      jdbc:mysql://localhost:3306/cafeteria_db?useSSL=false&serverTimezone=UTC
DB_USER:  root
DB_PASS:  Rajshri@1978
```

4. Model Layer — New Classes

4.1 Customer.java

Mapped to the customers table. Stores a customer's name and optional phone number. The findOrCreate pattern in CustomerDao ensures no duplicate customer rows are created for repeat visitors.

Method Signature	Returns	Description
`Customer(int, String, String)`	Constructor	Full constructor: customerId, name, phone.
`Customer(String, String)`	Constructor	Convenience constructor for new customers. Sets customerId = -1.
`getCustomerId()`	int	Primary key, or -1 before persistence.
`getName()`	String	Customer display name.
`getPhone()`	String	Contact number; may be null.

4.2 Order.java

Mapped to the orders table (header record). Holds a reference to the customerId, timestamp, and status. The items list is an in-memory list populated by OrderService — it is not stored as a column.

Method Signature	Returns	Description
`Order(int customerId)`	Constructor	New order constructor. Sets orderId = -1, status = PLACED.
`calculateSubtotal()`	double	Sums quantity × unitPrice across all in-memory items.
`addItem(OrderItem)`	void	Appends a line item to the in-memory items list.
`getStatus()`	String	PLACED, BILLED, or CANCELLED.
`getItems()`	List<OrderItem>	In-memory line items; populated by OrderService.

4.3 OrderItem.java

Mapped to the order_items table. Represents a single line in an order. Stores a snapshot of the unit_price at order time. The itemName field is fetched via a JOIN in OrderItemDao and is not a column in order_items.

Method Signature	Returns	Description

`getLineTotal()`	double	Returns quantity × unitPrice for this line item.
`getItemName()`	String	Denormalised name fetched via JOIN; not stored in order_items.
`getUnitPrice()`	double	Price snapshot captured at order time.
`getQuantity()`	int	Number of units ordered; must be > 0.

4.4 Bill.java

Mapped to the bills table. Has a one-to-one relationship with orders (UNIQUE constraint on order_id). Stores the pre-computed subtotal, tax amount, and final total.

Method Signature	Returns	Description
`Bill(int, double, double, double)`	Constructor	New bill constructor. Sets billId = -1, timestamp = now.
`getSubtotal()`	double	Pre-tax sum of all line items.
`getTaxAmount()`	double	Tax portion (5% of subtotal).
`getTotalAmount()`	double	Final amount = subtotal + taxAmount.
`getBillTimestamp()`	LocalDateTime	When the bill was generated.

5. DAO Layer — New Classes

All DAOs follow the same pattern: obtain a Connection from DBConnection.getInstance(), prepare a statement, execute, and map results to model objects. No business logic lives in a DAO.

5.1 CustomerDao.java

Method Signature	Returns	Description
`findByName(String)`	Customer	SELECT by name. Returns null if not found.
`insert(Customer)`	int	INSERT and return auto-generated customer_id.
`findOrCreate(String, String)`	int	Looks up by name; inserts a new record if absent. Returns customer_id.

5.2 OrderDao.java

Method Signature	Returns	Description
`insert(Order)`	int	INSERT order header; returns generated order_id.
`updateStatus(int, String)`	void	UPDATE status field for a given order_id.
`findAll()`	List<Order>	SELECT all orders, newest first.
`findById(int)`	Order	SELECT single order by primary key; null if not found.

5.3 OrderItemDao.java

Method Signature	Returns	Description
`insert(OrderItem)`	void	INSERT one line item. order_id must be set before calling.
`findByOrderId(int)`	List<OrderItem>	SELECT all items for an order; JOINs menu_items to populate itemName.

5.4 BillDao.java

Method Signature	Returns	Description
`insert(Bill)`	int	INSERT bill; returns generated bill_id.
`findByOrderId(int)`	Bill	SELECT bill for a given order; null if not yet billed.
`findAll()`	List<Bill>	SELECT all bills, newest first. Used by sales summary.

`getDailyTotal()`	double	SUM of total_amount for today's bills (WHERE DATE = CURDATE()).

6. Service Layer — New Classes

6.1 OrderService.java

Coordinates CustomerDao, OrderDao, and OrderItemDao to complete a full order placement. Also handles order retrieval for the history panel.

Method Signature	Returns	Description
`placeOrder(String, String, List<OrderItem>)`	Order	Validates input, runs the 3-step placement sequence, returns the saved Order.
`getAllOrders()`	List<Order>	Returns all orders with items loaded (calls OrderItemDao per order).
`getOrderById(int)`	Order	Returns one order with items loaded; null if not found.
`markAsBilled(int)`	void	Updates order status to BILLED. Called by BillService after bill creation.

Validation rules enforced by placeOrder():

- Customer name must not be blank.
- Items list must contain at least one entry.
- Every item's quantity must be greater than zero.

6.2 BillService.java

Handles bill calculation and persistence. The tax rate is defined as the constant TAX_RATE = 0.05 (5%). Change only this constant to update the rate across the whole application.

Method Signature	Returns	Description
`generateBill(Order)`	Bill	Calculates subtotal + tax, inserts bill, marks order as BILLED.
`getBillForOrder(int)`	Bill	Returns the bill for a given order; null if not yet billed.
`getAllBills()`	List<Bill>	Returns all bills, newest first.
`getDailyTotal()`	double	Today's total revenue in rupees.

Validation rules enforced by generateBill():

- Order must have at least one item (items list not empty).
- Order must not already be in BILLED status.

7. Database Schema

Run schema.sql in MySQL to create the database and all tables. The default credentials are username root, password Rajshri@1978.

7.1 customers

Column	Type / Constraint	Notes
customer_id	INT AUTO_INCREMENT PK	Primary key.
name	VARCHAR(100) NOT NULL	Customer display name.
phone	VARCHAR(20)	Optional contact number; may be NULL.

7.2 orders

Column	Type / Constraint	Notes
order_id	INT AUTO_INCREMENT PK	Primary key.
customer_id	INT NOT NULL FK→customers	The customer who placed this order.
order_timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP	Auto-set when the row is inserted.
status	VARCHAR(20) DEFAULT 'PLACED'	PLACED, BILLED, or CANCELLED.

7.3 order_items

Column	Type / Constraint	Notes
order_item_id	INT AUTO_INCREMENT PK	Primary key.
order_id	INT NOT NULL FK→orders	Parent order.
item_id	INT NOT NULL FK→menu_items	The menu item ordered.
quantity	INT NOT NULL CHECK (> 0)	Units ordered.
unit_price	DECIMAL(10,2) NOT NULL	Price snapshot at time of order.

7.4 bills

Column	Type / Constraint	Notes
bill_id	INT AUTO_INCREMENT PK	Primary key.

`order_id`	INT UNIQUE NOT NULL FK→orders	One bill per order (UNIQUE constraint).
`subtotal`	DECIMAL(10,2) NOT NULL	Sum of all line item totals before tax.
`tax_amount`	DECIMAL(10,2) NOT NULL	5% of subtotal.
`total_amount`	DECIMAL(10,2) NOT NULL	subtotal + tax_amount.
`bill_timestamp`	DATETIME DEFAULT CURRENT_TIMESTAMP	Auto-set on INSERT.

7.5 Entity Relationship Summary

The relationships between tables are:

- customers 1 ──< orders (one customer can have many orders)
- orders 1 ──< order_items (one order has one or more line items)
- menu_items 1 ──< order_items (one menu item can appear in many orders)
- orders 1 ── bills (one order has at most one bill — UNIQUE)

8. Setup and Compilation

8.1 Database Setup

Open a terminal and run:

```
mysql -u root -p < schema.sql
```

Enter the password Rajshri@1978 when prompted. This creates cafeteria_db, all six tables, and seeds default users and sample menu items.

8.2 Compile

```
javac -cp ".;mysql-connector-j-9.6.0.jar" -d out -sourcepath src src/Main.java
```

On Linux/macOS replace the semicolon (;) with a colon (:).

8.3 Run

```
java -cp ".;out;mysql-connector-j-9.6.0.jar" Main
```

8.4 Default Login Credentials

Role	Username	Password
Admin	admin	admin123
Operator	operator	operator123

9. Next Steps — Phase 4

Phase 4 (UI and Integration) will replace the three placeholder stub panels with fully functional Swing screens:

- PlaceOrderPanel.java — item selection table, quantity inputs, customer name field, Generate Bill button.
- OrderHistoryPanel.java — scrollable table of all orders with expandable line-item detail.
- SalesSummaryPanel.java — daily total revenue display, bill count, and per-category breakdown.

All three panels will use the OrderService and BillService classes built in this phase — no new DAO work is expected.